

Servicio Nacional de Aprendizaje SENA

Centro de Gestión de Mercados, Logística y Tecnologías de la Información

Tecnólogo en Análisis y Desarrollo de Software

GA5-220501095-AA1-EV01

Taller sobre Construcción del Prototipo del Software

Presentado por:

Gerson Jonnathan López Oviedo

Código de aprendiz: 3186647

Docente:

Arnaldo Alfonso Montiel Brun

Fase 2 — Planeación

Actividad de Proyecto 4

Bogotá D.C., 2025

Tabla de Contenidos

Introducción	3
Sección 1. Taller de Construcción del Prototipo	4
1.1 Categorías identificadas en los requerimientos	4
1.2 Paleta de colores y componentes	5
1.3 Pantalla 1 — Autenticación de usuario	6
1.4 Pantalla 2 — Registro de usuario	8
1.5 Pantalla 3 — Pantalla principal (Home)	10
1.6 Pantalla 4 — Notificación de errores	12
2. Link interactivo	17
3. Normalización del modelo lógico	17
4. Políticas de seguridad	19
Sección 2. Fundamentos de Calidad de Software	21
2.1 Definición de calidad de software	21
2.2 Concepto de usabilidad en software	23
2.3 Estándares de calidad de software	24
2.4 Tecnologías para implementar calidad de software	26
Conclusiones	28
Referencias	29

Introducción

El presente documento consolida la evidencia GA5-220501095-AA1-EV01, correspondiente al taller sobre construcción del prototipo de software para el proyecto **TRUEQUE MATCH**, desarrollado en el marco del Tecnólogo en Análisis y Desarrollo de Software del SENA.

TRUEQUE MATCH es una aplicación móvil y web orientada a facilitar el intercambio de bienes, servicios, conocimientos y experiencias entre usuarios sin mediación de dinero, promoviendo la economía colaborativa y el aprendizaje comunitario dentro del contexto colombiano.

La construcción de un prototipo constituye una fase determinante dentro del ciclo de vida del software. Según Sommerville (2016), los prototipos reducen la incertidumbre en los procesos de elicitación de requisitos, ya que permiten materializar visualmente aquello que en etapas tempranas solo existe como descripción textual o diagramática.

El taller se estructura en torno a cuatro pantallas fundamentales: autenticación de usuario, registro de datos personales, pantalla principal y notificación de errores, cuyo diseño toma como base los requerimientos funcionales (RF01 a RF14) y no funcionales (RNF01 a RNF08) de la propuesta técnica, asegurando que cada componente visual responda a una necesidad concreta del sistema.

Como parte del taller 2, la calidad del software es un aspecto transversal en todas las fases del ciclo de vida de un sistema de información. No se trata únicamente de que el software funcione correctamente, sino de que lo haga de manera eficiente, segura, mantenible y satisfactoria para el usuario final. En el contexto colombiano, la adopción de estándares internacionales de calidad cobra especial relevancia frente al crecimiento del sector tecnológico y las exigencias del mercado laboral en el país.

Esta investigación aborda cuatro conceptos fundamentales: la definición de calidad de software, el concepto de usabilidad, los estándares internacionales que regulan la calidad y las tecnologías disponibles para implementarla. Para su elaboración se consultaron al menos tres fuentes académicas y normativas reconocidas, buscando que los conceptos presentados sean aplicables directamente al desarrollo de **TRUEQUE MATCH**.

La investigación se apoya principalmente en la norma ISO/IEC 25010 (2011), en el referente teórico de Pressman y Maxim (2020) y en los lineamientos de Nielsen (1993) sobre usabilidad, complementados con normativas colombianas pertinentes al desarrollo de software y la protección de datos personales.

Sección 1. Taller de Construcción del Prototipo

El prototipado de software es la práctica de construir representaciones visuales de un sistema antes de su implementación definitiva. Según Nielsen (1993), la evaluación temprana de interfaces permite detectar problemas de usabilidad con un costo significativamente menor al que representaría corregirlos en etapas avanzadas del desarrollo. En esta sección se presenta el proceso de diseño de las cuatro pantallas principales de TRUEQUE MATCH, incluyendo la justificación de los componentes, la paleta de colores seleccionada y la organización lógica de los elementos de entrada.

1.1 Categorías Identificadas en los Requerimientos

Para organizar los componentes de cada pantalla de manera coherente, se realizó un proceso de categorización de los requerimientos funcionales y no funcionales del sistema. Esta práctica, sugerida por Pressman y Maxim (2020), permite agrupar elementos que comparten propósito funcional y así garantizar una disposición lógica en la interfaz.

Las categorías identificadas en los requerimientos de TRUEQUE MATCH son las siguientes:

Categoría	Requerimientos asociados	Pantalla relacionada
Autenticación	RF01, RF02, RNF03	Pantalla 1 — Login
Registro de usuario	RF01, RF03, RNF04	Pantalla 2 — Registro
Navegación principal	RF04, RF05, RF06, RF11	Pantalla 3 — Home
Validación y errores	RNF01, RNF03, RNF05	Pantalla 4 — Errores

1.2 Paleta de Colores y Componentes

La selección de colores en el diseño de interfaces es un factor determinante para la experiencia del usuario. Según las directrices de Material Design (Google, 2023) y los principios establecidos por Tidwell (2011), una paleta reducida favorece la coherencia visual y facilita la

identificación de elementos interactivos. Para TRUEQUE MATCH se definió un sistema cromático de tres colores base, considerando criterios de contraste, accesibilidad y representación de la identidad de la aplicación.

Color	Código HEX	Nombre asignado	Uso principal
Rojo	#C0392B	Rojo TM	Botones primarios, identidad de marca, estados de error
Negro	#1A1A1A	Negro base	Fondo principal, profundidad y contraste
Blanco cálido	#F5F0EB	Blanco cálido	Texto, etiquetas, elementos sobre fondo oscuro

Los componentes de interfaz comunes a todas las pantallas se estandarizaron en cuanto a tipografía, tamaño y color, siguiendo el principio de consistencia propuesto por Shneiderman et al. (2016). Se utilizó la fuente Bebas Neue para títulos y elementos de marca, mientras que Nunito se empleó para cuerpos de texto, etiquetas y mensajes del sistema, dado que ambas presentan alta legibilidad en pantallas de dispositivos móviles.

GA5-220501095-AA1-EV01 · PROTOTIPO DE SOFTWARE

TRUEQUE MATCH

Taller sobre Construcción del Prototipo del Software — Sección 1

Gerson Jonnathan López Oviedo · 3186647 · SENA — Tecnólogo Análisis y Desarrollo de Software

PALETA DE COLORES

Máximo 3 colores base · Sistema cromático del prototipo

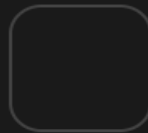
SISTEMA CROMÁTICO – TRUEQUE MATCH



ROJO TM

#C0392B

Acciones principales, CTA, identidad



NEGRO BASE

#1A1A1A

Fondo principal, profundidad



BLANCO CÁLIDO

#F5F0EB

Texto, contenido, contraste

COMPONENTES Y TIPOGRAFÍA

Bebas Neue

Títulos, encabezados y elementos de identidad de marca. Impacto visual.

Nunito 700

Subtítulos, etiquetas de campo, botones secundarios. Legibilidad alta.

Nunito 400

Texto de cuerpo, descripciones, mensajes de error e instrucciones.

Input Field

Fondo gris oscuro (#2C2C2C), borde sutil, radio 10–12px. Estado error: borde rojo.

Botón Primario

Fondo rojo (#C0392B), texto blanco en Bebas Neue, ancho completo, radio 12px.

Botón Secundario

Borde gris, fondo transparente, texto gris claro. Para acciones alternativas.

Card / Tarjeta

Fondo gris (#2C2C2C), radio 14px. Usada en ofertas, listas y resultados.

Banner Error

Fondo rojo semitransparente, borde rojo sólido, ícono de alerta. Radio 12px.

1.3 Pantalla 1 Autenticación de Usuario

La pantalla de autenticación constituye el primer punto de contacto entre el usuario y el sistema. Su objetivo principal es validar la identidad del usuario mediante la verificación de credenciales registradas previamente en la base de datos del sistema. De acuerdo con el requerimiento funcional RF02, el sistema debe implementar un mecanismo de inicio de sesión seguro con correo electrónico y contraseña.

Elementos de entrada.

Componente	Tipo	Validación aplicada	Requerimiento
Campo correo electrónico	Input texto	Formato email (expresión regular)	RF02, RNF03
Campo contraseña	Input password	Mínimo 8 caracteres, enmascarado	RF02, RNF03
Botón Ingresar	Botón primario	Activo solo si campos válidos	RF02
Enlace recuperar contraseña	Hipervínculo	Redirige a flujo de recuperación	RF02
Botón Crear cuenta	Botón secundario	Redirige a pantalla de registro	RF01

Organización lógica.

Los componentes se organizan en orden descendente según la secuencia natural de interacción: en primer lugar, el logo e identidad de la aplicación, seguido del campo de correo electrónico, el campo de contraseña, el enlace de recuperación, el botón de acción principal y finalmente la opción de registro. Este orden responde al modelo de flujo visual de arriba hacia abajo descrito por Lidwell et al. (2010).

Información adicional identificada.

Para completar el diseño funcional de esta pantalla se requiere definir si el sistema implementará autenticación mediante redes sociales (Google, Facebook), la política de bloqueo de cuenta tras intentos fallidos consecutivos y el comportamiento de la sesión en cuanto a duración y renovación automática del token de autenticación.

**ELEMENTOS DE ENTRADA**

- ▶ Campo correo electrónico (texto, validación formato email)
- ▶ Campo contraseña (enmascarado, mínimo 8 caracteres)
- ▶ Botón "Ingresar" — acción primaria (valida credenciales)
- ▶ Enlace recuperación de contraseña — acción secundaria
- ▶ Botón "Crear cuenta" — redirige a pantalla de registro
- ▶ Información faltante: ¿login con redes sociales? ¿2FA?

1.4 Pantalla 2 Registro de Usuario

La pantalla de registro permite que nuevos usuarios creen su cuenta en la plataforma ingresando sus datos personales básicos. Según el requerimiento RF01, el sistema debe permitir el registro de usuarios con validación de datos de identidad. Esta pantalla corresponde al primer paso de un flujo de registro de tres etapas, en la que se recopilan nombre, apellidos, número de cédula, fecha de nacimiento y correo electrónico.

Elementos de entrada.

<u>Campo</u>	<u>Tipo de dato</u>	<u>Restricción</u>	<u>Obligatorio</u>	<u>Requerimiento</u>
Nombre	VARCHAR(100)	Solo letras, mín. 2 caracteres	Sí	RF01, RF03
Apellidos	VARCHAR(100)	Solo letras, mín. 2 caracteres	Sí	RF01, RF03
Número de cédula	INT (8-10 dig.)	Solo dígitos, sin letras ni espacios	Sí	RF01
Fecha de nacimiento	DATE	Mayor de 18 años (validación de edad)	Sí	RF01, RNF04
Correo electrónico	VARCHAR(100)	Formato email, único en el sistema	Sí	RF01, RNF03
Aceptación de términos	Checkbox	Requerido para habilitar registro	Sí	RNF04

Organización lógica.

Los campos se presentan en el orden en que típicamente aparecen en documentos de identidad colombianos, facilitando así la transcripción de datos por parte del usuario. El indicador de progreso (pasos 1 de 3) brinda contexto sobre el proceso completo de registro, reduciendo la percepción de carga cognitiva que generan los formularios extensos (Krug, 2014).

Información adicional identificada.

Para la implementación completa de esta pantalla se requiere definir el tratamiento del campo de contraseña (criterios de complejidad y confirmación), el proceso de verificación del correo electrónico mediante enlace o código de confirmación, y la política de privacidad vinculada al tratamiento de datos personales conforme a la Ley 1581 de 2012.

2 PANTALLA DE REGISTRO DE USUARIO

9:41 ●●● WiFi

← **CREAR CUENTA**

Completa tus datos para unirte a la comunidad de trueque más grande de Colombia.

NOMBRE * **APELLIDOS ***

Ej: María Ej: López

NÚMERO DE CÉDULA *

10 dígitos

FECHA DE NACIMIENTO *

DD / MM / AAAA

CORREO ELECTRÓNICO *

usuario@correo.com

☒ Acepto los **Términos y condiciones** y la **Política de privacidad** de TRUEQUE MATCH.

SIGUIENTE →

ELEMENTOS DE ENTRADA

- ▶ Campo Nombre (texto, solo letras, requerido)
- ▶ Campo Apellidos (texto, solo letras, requerido)
- ▶ Campo Cédula (numérico, 8-10 dígitos, requerido)
- ▶ Campo Fecha de nacimiento (date picker, mayor de edad)
- ▶ Campo Correo (email, validación formato, requerido)
- ▶ Checkbox aceptación términos (requerido para continuar)
- ▶ Indicador de pasos (paso 1 de 3: datos personales)
- ▶ Información faltante: contraseña, foto de perfil

1.5 Pantalla 3 — Pantalla Principal (Home)

La pantalla principal es el centro de navegación de la aplicación y el punto de acceso a las funcionalidades principales del sistema. Desde esta vista, el usuario puede consultar ofertas recientes, explorar categorías de intercambio, realizar búsquedas y acceder a las demás secciones de la aplicación. Esta pantalla responde a los requerimientos funcionales RF04, RF05, RF06 y RF11, relacionados con la publicación, clasificación y visualización de ofertas.

Componentes de la pantalla.

Componente	Función	Requerimiento	Posición en pantalla
Header con logo y avatar	Identidad visual y acceso al perfil	RF03	Superior
Botón de notificaciones	Acceso al módulo de notificaciones	RF10	Superior derecha
Barra de búsqueda	Búsqueda global de ofertas	RF05, RF11	Media superior
Chips de categorías	Filtrado por tipo de intercambio	RF05	Media
Grid de ofertas recientes	Visualización de publicaciones activas	RF04, RF06	Centro
Menú de navegación inferior	Acceso a Inicio, Explorar, Publicar, Chat, Perfil	RF04 a RF09	Inferior fijo

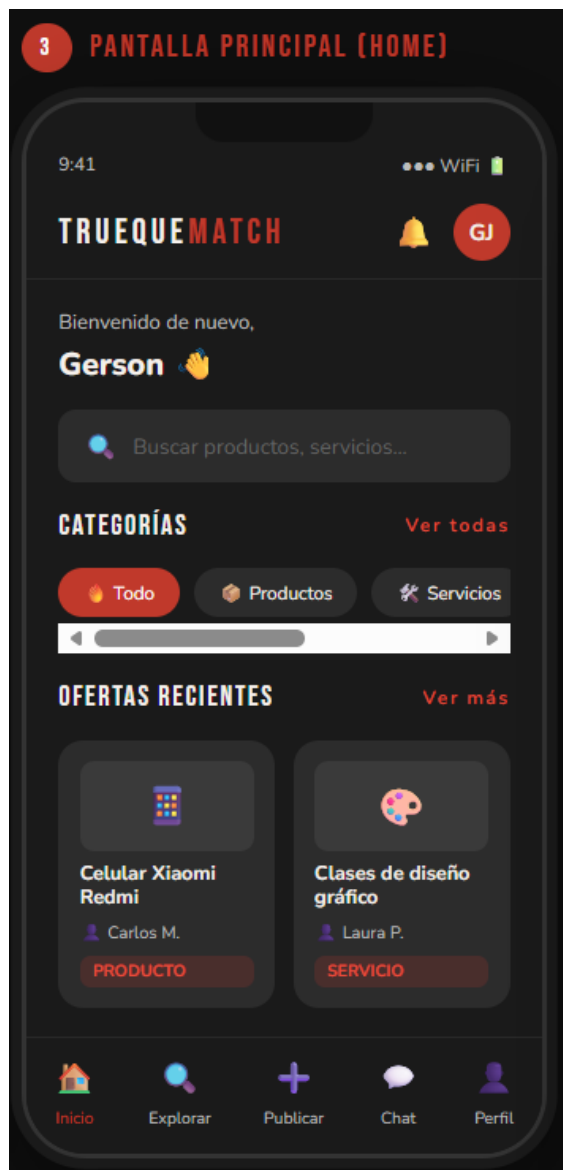
Organización lógica.

La pantalla sigue el patrón de diseño F-pattern identificado por Nielsen y Pernice (2010), en el que los elementos de mayor importancia se ubican en la zona superior izquierda de la pantalla, donde el ojo del usuario realiza su primer barrido visual. El menú de navegación inferior responde

al patrón de Tab Bar, ampliamente adoptado en aplicaciones móviles nativas por su accesibilidad y familiaridad para el usuario.

Información adicional identificada.

Para la implementación de esta pantalla se requiere definir el algoritmo de ordenamiento de las ofertas en el feed principal (por fecha, relevancia o proximidad geográfica), el comportamiento del sistema ante ausencia de conexión a internet y los criterios de paginación o carga infinita para el listado de ofertas.



COMPONENTES DE LA PANTALLA

- ▶ Header con logo, notificaciones y avatar de usuario
- ▶ Barra de búsqueda global (productos, servicios, usuarios)
- ▶ Chips de categorías (Productos, Servicios, Conocimientos, Experiencias)
- ▶ Grid de ofertas recientes en tarjetas (imagen, título, usuario, categoría)
- ▶ Menú inferior con 5 secciones: Inicio, Explorar, Publicar, Chat, Perfil
- ▶ Información faltante: filtros avanzados, feed de matches

1.6 Pantalla 4 Notificación de Errores al Usuario

La pantalla de notificación de errores tiene como propósito informar al usuario de manera clara y comprensible sobre las inconsistencias encontradas en los datos ingresados, permitiéndole identificar y corregir los campos afectados sin necesidad de reiniciar el proceso. Según el requerimiento no funcional RNF05, el sistema debe ofrecer una experiencia de usuario intuitiva, lo cual implica que los mensajes de error sean específicos, constructivos y ubicados en proximidad al campo que los origina.

Componentes del sistema de validación.

Elemento visual	Descripción	Requerimiento
Banner de error general	Alerta en la parte superior con resumen de errores encontrados	RNF01, RNF05
Lista resumen de validación	Muestra campos incorrectos (punto rojo) y correctos (punto verde)	RNF05
Campo con estado de error	Borde rojo y fondo semitransparente en campos que requieren corrección	RNF05
Mensaje de error contextual	Texto descriptivo bajo cada campo erróneo con indicación de corrección	RNF05
Campo con estado de éxito	Borde verde y mensaje de confirmación bajo campos correctamente diligenciados	RNF05

Organización lógica.

El diseño de esta pantalla aplica el principio de retroalimentación inmediata (immediate feedback) descrito por Nielsen (1994) como una de las diez heurísticas de usabilidad. Los errores se presentan en dos niveles: un nivel global mediante el banner superior que indica la cantidad total de errores, y un nivel local con los mensajes contextuales bajo cada campo. Esta doble

jerarquía permite al usuario comprender tanto el estado general del formulario como las acciones específicas requeridas en cada campo.

Información adicional identificada.

Para la implementación de este módulo se requiere definir el comportamiento de la validación en tiempo real frente a la validación al envío del formulario, el manejo de errores provenientes del servidor (por ejemplo, cédula ya registrada o correo duplicado) y la estrategia de accesibilidad para usuarios con discapacidad visual en lo referente a la lectura de mensajes de error por tecnologías de asistencia.

4

PANTALLA DE ERRORES Y VALIDACIÓN

9:41

WiFi

←

CREAR CUENTA


Se encontraron errores

Por favor revisa los campos marcados para continuar con tu registro.


Campos que requieren corrección

- La cédula ingresada no es válida
- Debes ser mayor de 18 años para registrarte
- Nombre y apellidos correctos ✓
- Correo electrónico válido ✓

NÚMERO DE CÉDULA *

1234AB


 La cédula solo puede contener números (8-10 dígitos)

FECHA DE NACIMIENTO *

15 / 03 / 2012


 Debes tener al menos 18 años para usar la app

CORREO ELECTRÓNICO *

geron@correo.com

✓ Correo electrónico válido

CORREGIR Y CONTINUAR

ELEMENTOS DE VALIDACIÓN

- ▶ Banner de error general visible al tope (rojo, ícono ⚠)
- ▶ Lista resumen con todos los errores e ítems correctos
- ▶ Campo erróneo: borde rojo, fondo rojo semitransparente
- ▶ Mensaje de error debajo de cada campo: descriptivo y claro
- ▶ Campo correcto: borde verde, mensaje de confirmación
- ▶ Botón "Corregir y continuar" siempre visible (no desaparece)
- ▶ Información faltante: errores de servidor / conexión

2. link interactivo prototipo [TRUEQUE MATCH — Prototipo Interactivo](#)

3. Normalización del Modelo Lógico

La normalización es un proceso sistemático aplicado al diseño de bases de datos relacionales con el propósito de eliminar redundancias, prevenir anomalías en las operaciones de inserción, actualización y eliminación, y garantizar la integridad estructural de los datos. Según Elmasri y Navathe (2016), la normalización consiste en descomponer las relaciones de una base de datos en estructuras más pequeñas y bien definidas, aplicando un conjunto progresivo de restricciones conocidas como formas normales.

En el modelo lógico de TRUEQUE MATCH se aplicaron las tres primeras formas normales (1FN, 2FN y 3FN), asegurando un diseño eficiente, coherente y escalable.

3.1 Primera Forma Normal (1FN)

Una relación se encuentra en primera forma normal cuando todos sus atributos contienen valores atómicos, es decir, valores indivisibles que no pueden descomponerse en unidades más pequeñas con significado propio dentro del contexto del modelo de datos (Codd, 1970). Adicionalmente, no deben existir grupos repetitivos ni atributos multivaluados dentro de una misma fila.

El modelo de TRUEQUE MATCH cumple con la 1FN dado que todos los campos de cada tabla contienen valores atómicos, no existen repeticiones de grupos de datos dentro de una misma fila y se definieron claves primarias únicas para identificar cada registro de manera inequívoca. Por ejemplo, en la tabla Trueque, los dos participantes del intercambio se representan mediante campos separados (id_usuario e id_usuario2) en lugar de almacenar múltiples valores en un solo campo.

3.2 Segunda Forma Normal (2FN)

Una relación se encuentra en segunda forma normal cuando está en 1FN y todos sus atributos no clave dependen funcionalmente de la totalidad de la clave primaria, eliminando así las dependencias parciales que pueden presentarse en tablas con claves compuestas (Date, 2004).

El modelo garantiza la 2FN mediante la separación de entidades conceptualmente distintas en tablas independientes. Entidades como Usuario, Oferta, Trueque y Evaluación fueron diseñadas de forma que ningún atributo dependa solamente de parte de la clave. Para representar relaciones de muchos a muchos entre Usuario y Oferta se implementó la tabla puente Usuario_ofertante, que almacena exclusivamente los identificadores de ambas entidades participantes, sin incorporar atributos que dependan parcialmente de alguno de ellos.

3.3 Tercera Forma Normal (3FN)

Una relación se encuentra en tercera forma normal cuando está en 2FN y no existen dependencias transitivas, es decir, ningún atributo no clave depende de otro atributo no clave dentro de la misma tabla (Elmasri y Navathe, 2016).

El modelo cumple con la 3FN al haber separado atributos que podrían generar dependencias transitivas en sus tablas correspondientes. Los campos clave acceso, comentario TDX y notificacionBK se encuentran en sus respectivas tablas (Administrador, Evaluación y Notificación), evitando duplicación de datos y manteniendo la coherencia semántica entre entidades y sus atributos.

Este proceso de normalización asegura que el sistema sea eficiente en almacenamiento y consultas, escalable para incorporar nuevas funcionalidades en versiones futuras y robusto frente a errores de integridad y duplicaciones de información.

4. Políticas de Seguridad

La seguridad de la información en sistemas de software se sustenta sobre tres pilares fundamentales conocidos como la tríada CIA: confidencialidad, integridad y disponibilidad (Stallings, 2017). Para TRUEQUE MATCH se definieron políticas específicas en cada uno de estos ámbitos, con el propósito de garantizar que el sistema sea confiable, trazable y resistente tanto ante fallos técnicos como ante accesos no autorizados.

El diseño de estas políticas responde además a lo establecido en la Ley 1581 de 2012 (Ley de Protección de Datos Personales de Colombia) y su Decreto Reglamentario 1377 de 2013, que obligan a los desarrolladores de sistemas de información a implementar medidas técnicas y organizativas para proteger los datos de los usuarios.

4.1 Integridad

La integridad de los datos garantiza que la información almacenada en el sistema sea exacta, completa y no haya sido alterada de manera no autorizada. Las medidas implementadas en el modelo de TRUEQUE MATCH para asegurar este pilar son las siguientes:

Se implementan claves primarias y foráneas en todas las tablas para asegurar la consistencia relacional e impedir el ingreso de registros huérfanos. Se aplican restricciones NOT NULL en campos críticos para evitar el almacenamiento de registros incompletos que pudieran comprometer la trazabilidad de las operaciones. Se utiliza AUTO_INCREMENT en todos los identificadores para prevenir la duplicidad de claves y garantizar la unicidad de cada registro. Finalmente, la normalización del modelo elimina las redundancias que podrían dar lugar a inconsistencias entre registros relacionados.

4.2 Confidencialidad

La confidencialidad asegura que la información sensible del sistema solo sea accesible para los usuarios y procesos debidamente autorizados. Para TRUEQUE MATCH se definieron las siguientes medidas:

Las contraseñas de usuarios y administradores se almacenan en la base de datos en formato cifrado mediante el algoritmo bcrypt, que incorpora un mecanismo de sal aleatoria (salt) que dificulta los ataques por tablas de hash precalculadas. Se recomienda implementar un modelo de control de acceso basado en roles (RBAC), diferenciando los privilegios del administrador del sistema frente a los del usuario estándar. El acceso a datos sensibles como correos electrónicos, evaluaciones, reportes y datos de cédula debe estar restringido mediante autenticación de sesión activa, impidiendo el acceso directo a los endpoints de la API sin token de autorización válido.

4.3 Disponibilidad

La disponibilidad garantiza que el sistema y sus datos sean accesibles para los usuarios autorizados en el momento en que los requieran. El requerimiento no funcional RNF02 establece un objetivo de disponibilidad del 99% de uptime para el sistema. Las medidas propuestas para alcanzar este objetivo son:

Se propone establecer un esquema de copias de seguridad periódicas de la base de datos con una frecuencia mínima diaria para ambientes de producción. El sistema debe contar con mecanismos de recuperación ante fallos, incluyendo replicación de la base de datos y procedimientos documentados para la restauración desde backups. El módulo de notificaciones y el módulo de reportes cumplen una función adicional como herramientas de monitoreo, al permitir la detección temprana de incidentes o anomalías que pudieran comprometer la continuidad del servicio.

Estas políticas forman parte integral del diseño estructural del sistema y deben complementarse con buenas prácticas de desarrollo seguro, monitoreo continuo y planes de respuesta ante incidentes durante las fases de implementación y mantenimiento del proyecto.

SECCIÓN 2. FUNDAMENTOS DE CALIDAD DE SOFTWARE

La calidad en el desarrollo de software es un campo de estudio que ha evolucionado significativamente desde las primeras décadas de la ingeniería de sistemas. Su importancia radica en que un software de baja calidad no solo genera pérdidas económicas para las organizaciones, sino que puede comprometer la seguridad de los usuarios y la confiabilidad de los procesos que soporta. La presente sección aborda los conceptos fundamentales que todo desarrollador debe conocer y aplicar en su práctica profesional.

2.1 Definición de Calidad de Software

La calidad de software es un concepto multidimensional que ha sido definido desde distintas perspectivas a lo largo del tiempo. Una de las definiciones más influyentes en la disciplina es la propuesta por Pressman y Maxim (2020), quienes la describen como la concordancia con los requisitos funcionales y de rendimiento explícitamente establecidos, con los estándares de desarrollo explícitamente documentados y con las características implícitas que se esperan de todo software desarrollado profesionalmente.

Esta definición introduce tres dimensiones complementarias. La primera hace referencia al cumplimiento de los requisitos funcionales, es decir, que el sistema realice efectivamente las

funciones para las que fue diseñado. La segunda apunta al cumplimiento de estándares documentados de desarrollo, lo cual implica que el proceso de construcción sigue metodologías y buenas prácticas reconocidas. La tercera corresponde a las características implícitas, aquellas que el usuario da por sentadas aunque no las haya especificado formalmente, como la estabilidad del sistema o la claridad de los mensajes de error.

Por su parte, la norma ISO/IEC 25010 (2011), conocida como el modelo SQuaRE (Systems and Software Quality Requirements and Evaluation), define la calidad del producto software como el grado en que dicho producto satisface las necesidades declaradas e implícitas de sus partes interesadas, aportando valor. Este modelo organiza la calidad en torno a ocho características principales: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. Cada una se subdivide en subcaracterísticas medibles que permiten evaluar de manera objetiva el nivel de calidad alcanzado.

En el contexto colombiano, el Ministerio de Tecnologías de la Información y las Comunicaciones (MinTIC) ha promovido la adopción del modelo de madurez CMMI (Capability Maturity Model Integration) como referente para las empresas del sector TI que aspiran a contratar con el Estado, según el Plan Nacional de Desarrollo 2022-2026. Este modelo define cinco niveles de madurez del proceso de desarrollo y establece criterios objetivos para evaluar la calidad del software producido.

Aplicando estos conceptos al proyecto TRUEQUE MATCH, la calidad del sistema no se limita a que los usuarios puedan publicar ofertas e intercambiar bienes. Abarca aspectos como el

tiempo de respuesta inferior a tres segundos (RNF01), la disponibilidad del 99% (RNF02), la seguridad de los datos personales conforme a la Ley 1581 de 2012 y la facilidad de uso de la interfaz para usuarios con distintos niveles de alfabetización digital.

2.2 Concepto de Usabilidad en Software

La usabilidad es una de las características de calidad más directamente perceptibles por el usuario final, y por ello ocupa un lugar central en el diseño de sistemas de información modernos. Nielsen (1993), considerado uno de los padres de la ingeniería de usabilidad, la define como el atributo de calidad que evalúa la facilidad de uso de las interfaces de usuario, descomponiéndola en cinco componentes medibles: capacidad de aprendizaje, eficiencia, memorabilidad, manejo de errores y satisfacción subjetiva.

La capacidad de aprendizaje hace referencia a qué tan fácil resulta para un usuario nuevo realizar tareas básicas la primera vez que interactúa con el sistema. La eficiencia mide la rapidez con que los usuarios experimentados realizan dichas tareas. La memorabilidad evalúa si un usuario que no ha usado el sistema por un tiempo puede retomar su uso sin necesidad de aprenderlo nuevamente. El manejo de errores considera la frecuencia y gravedad de los errores cometidos y la facilidad para recuperarse de ellos. Finalmente, la satisfacción subjetiva mide qué tan agradable resulta el uso del sistema.

La norma ISO/IEC 25010 (2011) incorpora la usabilidad como una de las ocho características principales del modelo de calidad del producto software, subdividiéndola en seis subcaracterísticas: reconocibilidad de adecuación, capacidad de aprendizaje, operabilidad, protección frente a errores del usuario, estética de la interfaz y accesibilidad. Esta última subcaracterística es de especial relevancia en Colombia, donde la Ley 1618 de 2013 establece la obligación de garantizar el pleno ejercicio de los derechos de las personas con discapacidad, incluyendo el acceso a tecnologías de la información.

En el ámbito del diseño de interfaces móviles, plataforma principal de TRUEQUE MATCH, Google (2023) establece en sus directrices de Material Design que los principios de usabilidad deben traducirse en decisiones concretas: tamaños mínimos de elementos táctiles de 48 x 48 píxeles, contraste mínimo de 4.5:1 entre texto y fondo, y tiempos de respuesta percibidos inferiores a 100 milisegundos para las interacciones inmediatas. Estos lineamientos fueron aplicados en el diseño del prototipo, que adopta una paleta de alto contraste, componentes de tamaño estándar y retroalimentación visual inmediata.

Shneiderman et al. (2016) complementan este enfoque con sus ocho reglas de oro del diseño de interfaces, entre las que destacan la consistencia en el diseño, la retroalimentación informativa al usuario, la prevención de errores y el soporte para el control interno del usuario. Estas reglas orientaron la organización de los componentes en las cuatro pantallas del prototipo de TRUEQUE MATCH, garantizando una experiencia coherente, predecible y satisfactoria.

2.3 Estándares de Calidad de Software

Los estándares de calidad de software son documentos normativos elaborados por organismos internacionales que establecen los requisitos, criterios y metodologías para garantizar que un producto o proceso de software alcance un nivel de calidad definido y verificable. Su adopción permite a las organizaciones desarrolladoras demostrar objetivamente la calidad de sus productos y alinear sus prácticas con referentes reconocidos internacionalmente.

El estándar más relevante en el campo de la calidad del producto software es la familia de normas ISO/IEC 25000, denominada SQuaRE. Esta familia reemplazó a los anteriores estándares ISO/IEC 9126 e ISO/IEC 14598 y se estructura en cinco divisiones: gestión de la calidad (ISO/IEC 2500n), modelo de calidad (ISO/IEC 2501n), medición de la calidad (ISO/IEC 2502n), requisitos de calidad (ISO/IEC 2503n) y evaluación de la calidad (ISO/IEC 2504n). La norma ISO/IEC 25010 (2011) es la más utilizada en la industria, pues define las características y subcaracterísticas de calidad aplicables tanto al producto software como a los sistemas en uso.

En el ámbito de los procesos de desarrollo, el estándar de referencia es la norma ISO/IEC 12207 (2017), que establece un marco de trabajo para los procesos del ciclo de vida del software, cubriendo desde la concepción del sistema hasta su retiro. Su aplicación en TRUEQUE MATCH se refleja en la estructuración del proyecto por fases claramente definidas: análisis, diseño, desarrollo, pruebas, implementación y mantenimiento.

El modelo CMMI, desarrollado por el Software Engineering Institute (SEI) de la Universidad Carnegie Mellon, es otro referente fundamental para la evaluación de la madurez de los procesos de desarrollo. Según Chrissis et al. (2011), define cinco niveles de madurez: inicial, gestionado, definido, gestionado cuantitativamente y en optimización. En Colombia, el MinTIC ha incentivado la certificación en niveles CMMI como criterio de habilitación para participar en licitaciones públicas del sector TI, impulsando su adopción en el país.

En el contexto colombiano también resulta relevante la Norma Técnica Colombiana NTC-ISO/IEC 27001 (2013), adoptada por ICONTEC, que establece los requisitos para los sistemas de gestión de seguridad de la información. Su aplicación es fundamental para garantizar la protección de los datos personales que maneja TRUEQUE MATCH, en cumplimiento de la Ley 1581 de 2012.

Estándar	Organismo	Ámbito de aplicación	Relevancia en el proyecto
ISO/IEC 25010:2011	ISO / IEC	Calidad del producto software	Define las 8 características de calidad del sistema
ISO/IEC 12207:2017	ISO / IEC	Procesos del ciclo de vida del software	Marco para las fases de desarrollo del proyecto
CMMI v2.0	ISACA / SEI	Madurez de procesos de desarrollo	Referente promovido por MinTIC para empresas TI en Colombia
NTC-ISO/IEC 27001:2013	ICONTEC / ISO	Gestión de seguridad de la información	Protección de datos personales conforme a Ley 1581 de 2012

Nota. Estándares de calidad de software aplicables al proyecto TRUEQUE MATCH. Elaboración propia con base en ISO (2011, 2017), ICONTEC (2013) y MinTIC (2022).

2.4 Tecnologías para Implementar Calidad de Software

La implementación de la calidad de software no depende únicamente de la adopción de estándares normativos, sino también del uso de herramientas tecnológicas que permitan verificar, medir y mejorar la calidad del código y del comportamiento del sistema de manera sistemática y automatizada. Pressman y Maxim (2020) clasifican estas herramientas en cuatro categorías: análisis estático, pruebas automatizadas, integración y entrega continua, y monitoreo en producción.

Las herramientas de análisis estático permiten examinar el código fuente sin ejecutarlo, identificando errores potenciales, vulnerabilidades de seguridad y violaciones de estándares de codificación. Para el stack tecnológico de TRUEQUE MATCH, basado en Node.js y React Native, las herramientas más relevantes son ESLint, que verifica convenciones de estilo y detecta errores comunes en JavaScript, y SonarQube, una plataforma de análisis de calidad compatible con múltiples lenguajes que genera informes sobre deuda técnica, cobertura de pruebas y puntos críticos de seguridad.

En el ámbito de las pruebas automatizadas, Jest es el framework de pruebas unitarias más utilizado en el ecosistema JavaScript y React Native. Permite crear pruebas automatizadas que verifican el comportamiento correcto de funciones y componentes individuales, facilitando la detección temprana de regresiones al modificar el código. Para pruebas de interfaz de usuario y flujos de extremo a extremo, Detox es la herramienta de referencia en React Native, ya que permite simular interacciones del usuario sobre un dispositivo real o emulado y verificar que la aplicación responde correctamente en todos los casos de uso definidos.

Las prácticas de integración y entrega continua, conocidas como CI/CD, son fundamentales para mantener la calidad del software a lo largo del tiempo. GitHub Actions, integrado en el repositorio de control de versiones del proyecto, permite ejecutar automáticamente las pruebas

unitarias, el análisis estático y la verificación de estándares cada vez que se sube un cambio al repositorio. Esta práctica, recomendada por el MinTIC (2022), reduce el riesgo de introducir defectos durante el desarrollo colaborativo.

Finalmente, en el monitoreo de la calidad en producción, Firebase Crashlytics ofrece una solución integrada con React Native que registra automáticamente los fallos que experimentan los usuarios en sus dispositivos, generando reportes en tiempo real. Esta herramienta se alinea directamente con el requerimiento no funcional RNF02 de TRUEQUE MATCH, que establece un objetivo de disponibilidad del 99%, ya que permite detectar y responder rápidamente ante cualquier degradación del servicio.

Herramienta	Categoría	Función principal	Aplicación en TRUEQUE MATCH
ESLint	Análisis estático	Verificación de estilo y errores en JavaScript	Revisión automática del código Node.js y React Native
SonarQube	Análisis estático	Métricas de calidad, seguridad y deuda técnica	Auditoría de calidad del código del backend y frontend
Jest	Pruebas automatizadas	Pruebas unitarias en JavaScript y React Native	Verificación de lógica de negocio y componentes
Detox	Pruebas automatizadas	Pruebas de interfaz de usuario extremo a extremo	Simulación de flujos de usuario en la app móvil
GitHub Actions	CI/CD	Automatización de pruebas y despliegue continuo	Ejecución automática de pruebas en cada commit
Firebase Crashlytics	Monitoreo en producción	Detección y reporte de fallos en tiempo real	Control de disponibilidad y corrección de errores en producción

Nota. Herramientas tecnológicas para la implementación de calidad en TRUEQUE MATCH. Elaboración propia con base en Pressman y Maxim (2020) y MinTIC (2022).

Conclusiones

La investigación desarrollada en esta sección permitió consolidar una comprensión integral de los fundamentos de la calidad de software y su aplicación concreta en el proyecto TRUEQUE MATCH. Los conceptos estudiados no son abstracciones teóricas desvinculadas de la práctica, sino referentes directamente aplicables a las decisiones de diseño, desarrollo y evaluación del sistema.

La definición de calidad propuesta por ISO/IEC 25010 (2011) y complementada por Pressman y Maxim (2020) deja en evidencia que la calidad es una propiedad multidimensional que trasciende el simple funcionamiento correcto del sistema. Para TRUEQUE MATCH, esto significa que el éxito del proyecto no se mide únicamente por la capacidad de procesar trueques, sino por su desempeño, seguridad, mantenibilidad y, sobre todo, por la satisfacción que genera en sus usuarios.

La usabilidad, entendida desde los principios de Nielsen (1993) y los lineamientos de Material Design (Google, 2023), se tradujo en decisiones concretas durante el diseño del prototipo: paleta de alto contraste, componentes de tamaño estándar, retroalimentación visual inmediata y mensajes de error contextuales. Estas decisiones responden igualmente a las exigencias de accesibilidad de la Ley 1618 de 2013 del marco normativo colombiano.

Finalmente, la adopción de estándares como ISO/IEC 25010, ISO/IEC 12207 y los lineamientos del MinTIC, junto con el uso de herramientas como ESLint, Jest, SonarQube, GitHub Actions y Firebase Crashlytics, configura un marco robusto para garantizar que TRUEQUE MATCH sea desarrollado con los niveles de calidad exigidos tanto por el mercado como por la normativa vigente en Colombia.

Referencias

Chrissis, M. B., Konrad, M., y Shrum, S. (2011). *CMMI for development: Guidelines for process integration and product improvement* (3.^a ed.). Addison-Wesley.

Google. (2023). *Material Design 3: Accessibility*. <https://m3.material.io/foundations/accessible-design/overview>

Instituto Colombiano de Normas Técnicas y Certificación. (2013). *NTC-ISO/IEC 27001: Tecnología de la información. Sistemas de gestión de la seguridad de la información*. ICONTEC.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387. <https://doi.org/10.1145/362384.362685>

Date, C. J. (2004). *An introduction to database systems* (8.^a ed.). Pearson Addison-Wesley.

Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of database systems* (7.^a ed.). Pearson.

Google. (2023). *Material Design 3: Color system*. <https://m3.material.io/styles/color/overview>

Krug, S. (2014). *Don't make me think, revisited: A common sense approach to web usability* (3.^a ed.). New Riders.

Lidwell, W., Holden, K., y Butler, J. (2010). *Universal principles of design* (2.^a ed.). Rockport Publishers.

López Oviedo, G. J. (2025). *Propuesta técnica y económica — TRUEQUE MATCH*. G.L. Software Solutions. SENA.

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 152-158). ACM. <https://doi.org/10.1145/191666.191729>

Nielsen, J., y Pernice, K. (2010). *Eyetracking web usability*. New Riders.

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., y Elmqvist, N. (2016). *Designing the user interface: Strategies for effective human-computer interaction* (6.^a ed.). Pearson.

Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson Education.

Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7.^a ed.). Pearson.

Tidwell, J. (2011). *Designing interfaces: Patterns for effective interaction design* (2.^a ed.). O'Reilly Media.

International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models*. ISO.

International Organization for Standardization. (2017). *ISO/IEC 12207: Systems and software engineering — Software life cycle processes*. ISO.

López Oviedo, G. J. (2025). *Propuesta técnica y económica — TRUEQUE MATCH*. G.L. Software Solutions. SENA.

Ministerio de Tecnologías de la Información y las Comunicaciones. (2022). *Guía de ingeniería de software para el sector público colombiano*. MinTIC. <https://www.mintic.gov.co>

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.

República de Colombia. (2012). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Congreso de la República.

República de Colombia. (2013). *Ley 1618 de 2013: Por medio de la cual se establecen las disposiciones para garantizar el pleno ejercicio de los derechos de las personas con discapacidad*. Congreso de la República.

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., y Elmqvist, N. (2016). *Designing the user interface: Strategies for effective human-computer interaction* (6.^a ed.). Pearson.